Racket Macro: (re)Match

```
(define-syntax (uncle-sams-holy-trinity stx)
  #''(list Aws Luke Xiao))
```

General Structure & Examples

```
(match '(1 2 3)
  [(list a b c)
   #:when (= 6 (+ a b c))
   'sum-is-six]
  [(list a b c) 'sum-is-not-six])
```

The match form has the syntax : **`(match val-expr clause …)`**

The available features of **`match`** include:

- **Variable Bindings**
- **Optional Conditional Expressions (ie. `#:when`)**
- **Multiple Clauses**

```
(match '(1 2)
  [(list a b c)
   #:when (= 6 (+ a b c))
   'sum-is-six]
  [(list a b)
   (match a
     [(? number?)
      'car-is-num])])
```

This example builds on the previous one by adding a **nested match**, which showcases the flexibility of the **`match`** macro.

Techniques & Decisions

- The `match` form is defined within a **`define-forms`** which expands to a `begin` expression that provides a number of match macros that use the same expander (**`go`**).

- Passed to the expander are:
  1. the full match expression,
  2. the expression being matched, and
  3. the list of clauses the parser and compiler need to work with.

- The **parser** checks the syntax of the `match` expression, and reforms its clauses using custom `structs` that can be passed to the compiler.

- The **compiler** compiles the body of the match expressions, and combines it with a compiled form of the clauses to get a fully compiled `match` expression. This loops in the fail condition, which is interestingly made into the else condition if there is one.

- This is then combined with outer fail expressions and it outputs.

Transforming the Input

Example of a simple `match` expression

```
(match (+ 1 2)
  [(list a b) (+ a b)]
  [x x])
```

Output from parsing the clauses

```
((Row ((Pair (Var a)
             (Pair (Var b) (Null))))
      <expr>)
 (Row ((Var x))
      <expr>))
```

Output from compiling the body

```
(let ((next17
       (λ ()
         (let* ((f18
                 (lambda ()
                   (syntax-parameterize
                    ((fail (make-rename-transformer (quote-syntax fail2))))
                    (let ((x temp1)) (let () x))))))
           (cond
            ((pair? temp1)
             (let ((unsafe-car21 (unsafe-car temp1))
                   (unsafe-cdr22 (unsafe-cdr temp1)))
               (cond
                ((pair? unsafe-cdr22)
                 (let ((unsafe-car25 (unsafe-car unsafe-cdr22))
                       (unsafe-cdr26 (unsafe-cdr unsafe-cdr22)))
                   (cond
                    ((null? unsafe-cdr26)
                     (syntax-parameterize
                      ((fail (make-rename-transformer (quote-syntax f18))))
                      (let ((b unsafe-car25))
                        (let ((a unsafe-car21)) (let () (+ a b))))))
                    (else (f18)))))
                (else (f18)))))
            (else (f18)))))))
  (case temp1 (else (next17))))
```

< Final fully expanded macro (as outputted by `match`. Preceding further expansion of used macros ie. `let*`)

```
(let ((temp1 (+ 1 2)))
  (let ((fail2
         (λ ()
           (match:error
            temp1
            (syntax-srclocs (quote-syntax srcloc))
            'match))))
    (let ((next3
           (λ ()
             (let* ((f4
                     (lambda ()
                       (syntax-parameterize
                        ((fail
                          (make-rename-transformer (quote-syntax fail2))))
                        (let ((x temp1)) (let () x))))))
               (cond
                ((pair? temp1)
                 (let ((unsafe-car7 (unsafe-car temp1))
                       (unsafe-cdr8 (unsafe-cdr temp1)))
                   (cond
                    ((pair? unsafe-cdr8)
                     (let ((unsafe-car11 (unsafe-car unsafe-cdr8))
                           (unsafe-cdr12 (unsafe-cdr unsafe-cdr8)))
                       (cond
                        ((null? unsafe-cdr12)
                         (syntax-parameterize
                          ((fail (make-rename-transformer (quote-syntax f4))))
                          (let ((b unsafe-car11))
                            (let ((a unsafe-car7)) (let () (+ a b))))))
                        (else (f4)))))
                    (else (f4)))))
                (else (f4)))))))
      (case temp1 (else (next3))))))
```